

Formalizing the 3-State Approximate Majority Population Protocol in Lean 4

February 2026

Abstract

We present a formalization in Lean 4 (with Mathlib) of the 3-state approximate majority population protocol introduced by Angluin, Aspnes, and Eisenstat (2008). The formalization covers the protocol’s state space, deterministic transition function, configuration representation with a built-in population size invariant, single-step dynamics, and several key structural invariants proved by exhaustive case analysis. We also lay the groundwork for a probabilistic treatment by defining the uniform random scheduler as a probability mass function. All deterministic results are machine-checked without axioms; the probabilistic components include clearly marked stubs for future work.

Contents

1	Introduction	1
2	The Protocol	1
2.1	States and Transition Rules	1
2.2	Configurations	2
3	Lean Formalization Strategy	2
3.1	Design Decisions	2
3.2	Project Structure	2
4	Type Definitions	2
4.1	State	2
4.2	Transition Function	3
4.3	Configuration	3
5	Step Function	3
6	Invariant Proofs	4
6.1	Population Size	4
6.2	One-Way Property	4
6.3	Gap Bounded Change	4
6.4	Absorbing States	4
6.5	Last Agent Survival	5
7	Probabilistic Framework	5
7.1	Uniform Random Scheduler	5
7.2	One-Step Distribution	5
7.3	Markov Chain (Stub)	5

8	Proof Techniques	6
9	Challenges and Future Work	6
9.1	Challenges Encountered	6
9.2	Future Work	6

1 Introduction

Population protocols [1] model distributed computation among a collection of finite-state agents that interact in pairs chosen by an adversary or random scheduler. The *approximate majority* protocol of Angluin, Aspnes, and Eisenstat [2] achieves consensus using only **three states**—opinion x , blank b , and opinion y —with expected convergence time $O(n \log n)$ under a uniform random scheduler.

This document describes a Lean 4 formalization of the protocol’s core definitions, deterministic invariants, and the beginnings of a probabilistic framework. The formalization is structured as a Lake project called `PopProto` with `Mathlib` as a dependency.

2 The Protocol

2.1 States and Transition Rules

The protocol uses three states:

$$Q = \{x, b, y\}$$

where x and y represent opposing opinions and b is a “blank” (undecided) state.

When two agents interact, one is designated the *initiator* and the other the *responder*. The key asymmetry is:

One-way property: The initiator’s state never changes.

The full transition table $\delta(i, r) = (i', r')$ is:

Initiator i	Responder r	Initiator i'	Responder r'
x	x	x	x
x	b	x	x
x	y	x	b
b	x	b	x
b	b	b	b
b	y	b	y
y	x	y	b
y	b	y	y
y	y	y	y

Intuitively: a blank responder adopts the initiator’s opinion; two agents with opposing opinions “neutralize” the responder to blank; a blank initiator does nothing.

2.2 Configurations

A *configuration* of a population of n agents is a triple $(x_c, b_c, y_c) \in \mathbb{N}^3$ satisfying $x_c + b_c + y_c = n$.

Decision	Choice	Rationale
Config representation	Struct with proof field	Sum invariant by construction
Population size	Type parameter <code>Config n</code>	Type-level separation
Counts	<code>Nat</code> (not <code>Fin</code>)	Simpler arithmetic; <code>omega</code>
Proof strategy	Exhaustive case split	Only 9 cases; closes by <code>rfl/omega</code>
Probability	PMF from Mathlib	Ready-made PMF type

Table 1: Key design decisions.

3 Lean Formalization Strategy

3.1 Design Decisions

3.2 Project Structure

The formalization is organized as follows:

- `PopProto/State.lean` — The inductive type `State`
- `PopProto/Transition.lean` — The transition function δ
- `PopProto/Config.lean` — Configuration type with sum constraint
- `PopProto/Step.lean` — Single-step configuration update
- `PopProto/Invariant/` — Five invariant proof modules
- `PopProto/Probability/` — Scheduler, step distribution, Markov chain stub

4 Type Definitions

4.1 State

The state type is a simple inductive with three constructors:

```

1 inductive State
2   | x  -- opinion X
3   | b  -- blank (undecided)
4   | y  -- opinion Y
5   deriving DecidableEq, Repr

```

We derive `DecidableEq` (for case splits in `if` expressions) and provide a `Fintype` instance (the universe has exactly 3 elements).

4.2 Transition Function

The transition function is defined by pattern matching on all 9 cases:

```

1 def delta : State → State → State → State
2   | (.x, .x) => (.x, .x)
3   | (.x, .b) => (.x, .x)
4   | (.x, .y) => (.x, .b)
5   | (.b, .x) => (.b, .x)
6   | (.b, .b) => (.b, .b)
7   | (.b, .y) => (.b, .y)
8   | (.y, .x) => (.y, .b)
9   | (.y, .b) => (.y, .y)
10  | (.y, .y) => (.y, .y)

```

The one-way property is an immediate consequence:

```

1 @[simp]
2 theorem deltafst (p : State  $\times$  State) :
3   (delta p).1 = p.1 := by
4     rcases p with  $\langle i, r \rangle$ ; cases i <;> cases r <;> rfl

```

4.3 Configuration

A configuration bundles three natural-number counts with a proof that they sum to the population size n :

```

1 structure Config (n :  $\mathbb{N}$ ) where
2   x_count :  $\mathbb{N}$ 
3   b_count :  $\mathbb{N}$ 
4   y_count :  $\mathbb{N}$ 
5   sum_eq : x_count + b_count + y_count = n

```

By carrying `sum_eq` as a field, the population-size invariant is enforced *by construction*: any function that produces a `Config n` must supply a proof. This eliminates an entire class of potential errors and makes the invariant trivially true (Theorem 6.1).

Helper definitions include:

- `gap` : $\mathbb{Z}$ — the signed difference $x_c - y_c$
- `opinionated` : $\mathbb{N}$ — the count $x_c + y_c$
- `allX`, `allY`, `allB` — consensus predicates
- `countOf` : $\text{State} \rightarrow \mathbb{N}$ — state-indexed lookup

5 Step Function

The function `Config.step` applies one interaction to a configuration. Given an initiator state i and responder state r , it:

1. Checks feasibility (enough agents of the required types).
2. Computes the new responder state $r' = \delta(i, r)_2$.
3. Adjusts the counts: `count(r)` decreases by 1 and `count(r')` increases by 1 (when $r \neq r'$).
4. Returns the new configuration wrapped in `Option`, or `none` if infeasible.

The function is defined by explicit pattern matching on all 9 cases. For each non-trivial case, the proof obligation $x' + b' + y' = n$ is closed by `omega` using the hypothesis `c.sum_eq` and the feasibility guard.

A total wrapper `stepOrElse` returns the original configuration when the interaction is infeasible:

```

1 def stepOrElse (c : Config n) (i r : State) : Config n :=
2   (c.step i r).getD c

```

6 Invariant Proofs

All invariant proofs follow the same pattern:

1. Unfold `step` to expose the 9-way match.
2. Apply `cases i <;> cases r` to split into 9 goals.
3. Close trivial goals with `simp_all` and `omega`.
4. For non-trivial goals, destructure the `Option.some` hypothesis and simplify.

6.1 Population Size

Theorem 6.1 (Population Size Invariant). *For any configuration c of size n and any interaction (i, r) , if $c' = \text{step}(c, i, r)$, then $c'.x + c'.b + c'.y = n$.*

This is trivially true because `Config n` enforces $x + b + y = n$ by construction. The proof is simply `c'.sum_eq`.

6.2 One-Way Property

Theorem 6.2 (One-Way Property). *For any states i, r , we have $\delta(i, r)_1 = i$.*

Proved by `cases i <;> cases r <;> rfl` — each of the 9 cases is definitionally equal.

6.3 Gap Bounded Change

Theorem 6.3 (Gap Invariant). *If $c \xrightarrow{(i,r)} c'$, then $|u' - u| \leq 1$ where $u = x_c - y_c$ and $u' = x_{c'} - y_{c'}$ (computed over $\mathbb{Z}$).*

This is proved by exhaustive 9-case analysis. In each case, at most one of x_c or y_c changes by ± 1 , so the gap changes by at most 1.

6.4 Absorbing States

Theorem 6.4 (Absorbing Configurations). *The configurations `all-x`, `all-y`, and `all-b` are absorbing: if c is one of these, then $\text{stepOrSelf}(c, i, r) = c$ for all i, r .*

When all agents share the same state, every interaction is between two agents of that state, and $\delta(s, s) = (s, s)$ for all s .

Theorem 6.5 (Unreachability of All-Blank). *If c has at least one opinionated agent ($x_c + y_c > 0$), then for any step $c \rightarrow c'$, the configuration c' also has at least one opinionated agent. In particular, `all-b` is unreachable.*

The proof uses the `hasOpinion_preserved` lemma: the last non-blank agent can only interact with blanks, and in every such interaction (e.g., $(x, b) \rightarrow (x, x)$), the blank is converted rather than the opinionated agent being eliminated.

6.5 Last Agent Survival

Theorem 6.6 (Last Agent). *If $x_c \geq 1$ and $y_c = 0$, then after any step, $x_{c'} \geq 1$ and $y_{c'} = 0$. Symmetrically for y .*

With no opposing agents, the only possible interactions involving an x -agent are (x, x) and (x, b) , both of which preserve or increase the x -count.

7 Probabilistic Framework

7.1 Uniform Random Scheduler

Under the uniform random scheduler, an ordered pair of distinct agents (a_i, a_j) is selected uniformly from $n(n-1)$ possibilities. The probability of an interaction of type (s_1, s_2) is:

$$\Pr[(s_1, s_2)] = \begin{cases} \frac{c(s_1) \cdot (c(s_1) - 1)}{n(n-1)} & \text{if } s_1 = s_2, \\ \frac{c(s_1) \cdot c(s_2)}{n(n-1)} & \text{if } s_1 \neq s_2. \end{cases}$$

We define `interactionCount` and `interactionProb` in `Scheduler.lean`. The key identity

$$\sum_{s_1 \in Q} \sum_{s_2 \in Q} \text{interactionCount}(c, s_1, s_2) = n(n-1)$$

is stated as `sum_interactionCount`. This identity follows from $(x + b + y)^2 - (x + b + y) = n^2 - n = n(n-1)$, but its formal proof requires careful handling of natural-number subtraction (since $a \cdot (a-1)$ in $\mathbb{N}$ truncates to 0 when $a = 0$). This proof is currently left as a `sorry` stub.

The PMF construction `interactionPMF` wraps the probabilities into Mathlib’s PMF (`State × State`) type.

7.2 One-Step Distribution

The one-step distribution `stepDist` composes the scheduler PMF with the deterministic step function via `PMF.map`:

```
1 noncomputable def stepDist (c : Config n)
2   (hn : n ≥ 2) : PMF (Config n) :=
3   PMF.map (fun p => c.stepOrSelf p.1 p.2)
4   (c.interactionPMF hn)
```

7.3 Markov Chain (Stub)

The file `MarkovChain.lean` outlines how to view the protocol as a discrete-time Markov chain using Mathlib’s `ProbabilityTheory.Kernel` framework. Constructing the transition kernel and proving convergence properties are left as future work.

8 Proof Techniques

The formalization relies heavily on a small set of Lean 4 / Mathlib tactics:

`cases i <;> cases r` Splits into 9 subgoals, one per (i, r) pair. Most transition-level properties reduce to definitional equality after this split.

`simp_all` Simplifies goals using hypotheses and `@[simp]` lemmas. Particularly effective after case-splitting, when the match arms of `step` have been reduced.

`omega` Decides linear natural-number and integer arithmetic. Closes sum-preservation goals, count bounds, and gap-change bounds.

`split_ifs` Case-splits on `if-then-else` expressions remaining after partial simplification.

`obtain` Destructures existential hypotheses produced when `simp_all` processes the `Option.some` constructor in step results.

9 Challenges and Future Work

9.1 Challenges Encountered

1. **Natural-number subtraction.** The expression $a \cdot (a-1)$ in $\mathbb{N}$ introduces truncation artifacts. The tactic `omega` handles linear arithmetic over $\mathbb{N}$ well, but the nonlinear identity $\sum c_i(c_i-1) + \sum_{i \neq j} c_i c_j = n(n-1)$ required for the scheduler’s sum-to-1 proof resists both `omega` (nonlinear) and `nlinarith` (Nat subtraction). Converting via `zify` is a promising approach but requires careful handling of `Nat.cast` applied to subtraction terms.
2. **Struct field unfolding.** When `step` returns `some {x_count := ..., ...}`, the hypothesis after `simp_all` takes the form $\exists h, \{\dots\} = c'$, requiring `obtain` to extract the proof and substitute.

3. **Linter noise.** Automated proofs using chains like `cases i <;> cases r <;> simp_all <;> ...` produce linter warnings about unused or unreachable tactics on branches that close early. We suppress these with targeted `set_option` directives.

9.2 Future Work

1. **Complete the scheduler sum-to-1 proof.** The algebraic identity is straightforward on paper; formalizing it requires either working in $\mathbb{Z}$ (via `zify`) or establishing auxiliary lemmas about $a \cdot (a - 1)$ in $\mathbb{N}$.
2. **Construct the Markov transition kernel.** Lift `stepDist` into Mathlib’s `ProbabilityTheory.Kernel` framework by equipping `Config n` with the discrete σ -algebra.
3. **Convergence to consensus.** Prove that, starting from any configuration with $x_c + y_c > 0$, the protocol reaches all- x or all- y with probability 1.
4. **Expected convergence time.** Formalize the $O(n \log n)$ bound from [2] using a potential-function argument and optional stopping times.
5. **Correctness of majority.** Show that if the initial majority is x (i.e., $x_0 > y_0$), then the protocol converges to all- x with probability $1 - o(1)$ as $n \rightarrow \infty$.

References

- [1] D. Angluin, J. Aspnes, Z. Diamadi, M. J. Fischer, and R. Peralta. Computation in networks of passively mobile finite-state sensors. *Distributed Computing*, 18(4):235–253, 2006.
- [2] D. Angluin, J. Aspnes, and D. Eisenstat. A simple population protocol for fast robust approximate majority. *Distributed Computing*, 21(2):87–102, 2008.